Introsort

Introsort or **introspective sort** is a hybrid sorting algorithm that provides both fast average performance and (asymptotically) optimal worst-case performance. It begins with quicksort, it switches to heapsort when the recursion depth exceeds a level based on (the logarithm of) the number of elements being sorted and it switches to insertion sort when the number of elements is below some threshold. This combines the good parts of the three algorithms, with practical performance comparable to quicksort on typical data sets and worst-case O(n log n) runtime due to the heap sort. Since the three algorithms it uses are comparison sorts, it is also a comparison sort.

Introsort

Class	Sorting algorithm
Data structure	Array
Worst-case performance	O(n log n)
Average performance	O(n log n)
Optimal	yes

Introsort was invented by David Musser in Musser (1997), in which he also introduced introselect, a hybrid selection algorithm based on quickselect (a variant of quicksort), which falls back to median of medians and thus provides worst-case linear complexity, which is optimal. Both algorithms were introduced with the purpose of providing generic algorithms for the C++ Standard Library which had both fast average performance and optimal worst-case performance, thus allowing the performance requirements to be tightened.[1] Introsort is in-place and a non-stable algorithm.

Pseudocode

If a heapsort implementation and partitioning functions of the type discussed in the quicksort article are available, the introsort can be described succinctly as

```
procedure sort(A : array):
    maxdepth ← ⌊log2(length(A))⌋ × 2
    introsort(A, maxdepth)

procedure introsort(A, maxdepth):
    n ← length(A)
    if n < 16:
        insertionsort(A)
    else if maxdepth = 0:
        heapsort(A)
    else:
        p ← partition(A)  // assume this function does pivot selection, p is the final position of the
pivot
        introsort(A[1:p-1], maxdepth - 1)
        introsort(A[p+1:n], maxdepth - 1)
```

The factor 2 in the maximum depth is arbitrary; it can be tuned for practical performance. $A[i:j]$ denotes the array slice of items i to j including both $A[i]$ and $A[j]$. The indices are assumed to

start with 1 (the first element of the A array is A[1]).

Analysis

In quicksort, one of the critical operations is choosing the pivot: the element around which the list is partitioned. The simplest pivot selection algorithm is to take the first or the last element of the list as the pivot, causing poor behavior for the case of sorted or nearly sorted input. Niklaus Wirth's variant uses the middle element to prevent these occurrences, degenerating to $O(n^2)$ for contrived sequences. The median-of-3 pivot selection algorithm takes the median of the first, middle, and last elements of the list; however, even though this performs well on many real-world inputs, it is still possible to contrive a *median-of-3 killer* list that will cause dramatic slowdown of a quicksort based on this pivot selection technique.

Musser reported that on a median-of-3 killer sequence of 100,000 elements, introsort's running time was 1/200 that of median-of-3 quicksort. Musser also considered the effect on caches of Sedgewick's delayed small sorting, where small ranges are sorted at the end in a single pass of insertion sort. He reported that it could double the number of cache misses, but that its performance with double-ended queues was significantly better and should be retained for template libraries, in part because the gain in other cases from doing the sorts immediately was not great.

Implementations

Introsort or some variant is used in a number of standard library sort functions, including some C++ sort implementations.

The June 2000 SGI C++ Standard Template Library stl_algo.h (http://www.sgi.com/tech/stl/stl_algo.h) implementation of unstable sort uses the Musser introsort approach with the recursion depth to switch to heapsort passed as a parameter, median-of-3 pivot selection and the Knuth final insertion sort pass for partitions smaller than 16.

The GNU Standard C++ library is similar: uses introsort with a maximum depth of $2\times\log_2 n$, followed by an insertion sort on partitions smaller than 16.[2]

LLVM libc++ also uses introsort with a maximum depth of $2\times\log_2 n$, however the size limit for insertion sort is different for different data types (30 if swaps are trivial, 6 otherwise). Also, arrays with sizes up to 5 are handled separately.[3] Kutenin (2022) provides an overview for some changes made by LLVM, with a focus on the 2022 fix for quadraticness.[4]

The Microsoft .NET Framework Class Library, starting from version 4.5 (2012), uses introsort instead of simple quicksort.[5]

Go uses a modification of introsort: for slices of 12 or less elements it uses insertion sort, and for larger slices it uses pattern-defeating quicksort and more advanced median of three medians for

pivot selection.[6] Prior to version 1.19 it used shell sort for small slices.

Java, starting from version 14 (2020), uses a hybrid sorting algorithm that uses merge sort for highly structured arrays (arrays that are composed of a small number of sorted subarrays) and introsort otherwise to sort arrays of ints, longs, floats and doubles.[7]

Variants

pdqsort

Pattern-defeating quicksort (pdqsort) is a variant of introsort developed by Orson Peters, incorporating the following improvements:[8]

- Median-of-three pivoting,
- "BlockQuicksort" partitioning technique to mitigate branch misprediction penalties,
- Linear time performance for certain input patterns (adaptive sort),
- Use element shuffling on bad cases before trying the slower heapsort.
- Improved adaptivity for low-cardinality inputs

Pdqsort is used by Boost,[9] GAP, Rust,[10] and Zig.[11]

fluxsort

fluxsort is a stable variant of introsort incorporating the following improvements:[12]

- branchless sqrt(n) pivoting
- Flux partitioning technique for stable partially-in-place partitioning
- Significantly improved smallsort by utilizing branchless bi-directional parity merges
- A fallback to quadsort, a branchless bi-directional mergesort, significantly increasing adaptivity for ordered inputs

Improvements introduced by fluxsort and its unstable variant, crumsort, were adopted by crumsort-rs, glidesort, ipnsort, and driftsort. The overall performance increase on random inputs compared to pdqsort is around 50%.[13][14][15][16][17]

References

1. "Generic Algorithms (http://www.cs.rpi.edu/~musser/gp/algorithms.html)", David Musser
2. libstdc++ Documentation: Sorting Algorithms (https://gcc.gnu.org/onlinedocs/libstdc++/libstdc++-html-USERS-4.4/a01027.html)
3. libc++ source code: sort (https://github.com/llvm/llvm-project/blob/368faacac7525e538fa6680aea74e19a75e3458d/libcxx/include/__algorithm/sort.h#L272)

4. Kutenin, Danila (20 April 2022). "Changing std::sort at Google's Scale and Beyond" (https://danlark.org/2022/04/20/changing-stdsort-at-googles-scale-and-beyond/comment-page-1). *Experimental chill*.
5. Array.Sort Method (Array) (http://msdn.microsoft.com/en-us/library/6tf1f0bc(v=vs.110).aspx)
6. Go 1.20.3 source code (https://github.com/golang/go/blob/go1.20.3/src/sort/zsortfunc.go#L61)
7. Java 14 source code (https://github.com/openjdk/jdk/blob/jdk-14-ga/src/java.base/share/classes/java/util/DualPivotQuicksort.java#L178)
8. Peters, Orson R. L. (2021). "orlp/pdqsort: Pattern-defeating quicksort" (https://github.com/orlp/pdqsort). *GitHub*. arXiv:2106.05123 (https://arxiv.org/abs/2106.05123).
9. Lammich, Peter (2020). *Efficient Verified Implementation of Introsort and Pdqsort*. IJCAR 2020: Automated Reasoning. Vol. 12167. pp. 307–323. doi:10.1007/978-3-030-51054-1_18 (https://doi.org/10.1007%2F978-3-030-51054-1_18).
10. "slice.sort_unstable(&mut self)" (https://doc.rust-lang.org/std/primitive.slice.html#method.sort_unstable). *Rust*. "The current algorithm is based on pattern-defeating quicksort by Orson Peters, which combines the fast average case of randomized quicksort with the fast worst case of heapsort, while achieving linear time on slices with certain patterns. It uses some randomization to avoid degenerate cases, but with a fixed seed to always provide deterministic behavior."
11. "pdq.zig" (https://github.com/ziglang/zig/blob/a0ec4e270e680960290642468f6df3ce7e7d7664/lib/std/sort/pdq.zig). *GitHub*. "number of allowed imbalanced partitions before switching to heap sort."
12. van den Hoven, Igor (2021). "fluxsort" (https://github.com/scandum/fluxsort). *GitHub*.
13. van den Hoven, Igor (2022). "crumsort" (https://github.com/scandum/crumsort). *GitHub*.
14. Tiselice, Dragoș (2022). "crumsort-rs" (https://github.com/google/crumsort-rs). *GitHub*.
15. Peters, Orson (2023). "Glidesort: Efficient In-Memory Adaptive Stable Sorting on Modern Hardware" (https://archive.fosdem.org/2023/schedule/event/rust_glidesort/).
16. Bergdoll, Lukas (2024). "ipnsort: an efficient, generic and robust unstable sort implementation" (https://github.com/Voultapher/sort-research-rs/blob/main/writeup/ipnsort_introduction/text.md).
17. Bergdoll, Lukas (2024). "driftsort: an efficient, generic and robust stable sort implementation" (https://github.com/Voultapher/sort-research-rs/blob/main/writeup/driftsort_introduction/text.md).

General

- Musser, David R. (1997). "Introspective Sorting and Selection Algorithms" (https://web.archive.org/web/20230307185457/http://www.cs.rpi.edu/~musser/gp/introsort.ps). *Software: Practice and Experience*. **27** (8): 983–993. doi:10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-# (https://doi.org/10.1002%2F%28SICI%291097-024X%28199708%2927%3A8%3C983%3A%3AAID-SPE117%3E

3.0.CO%3B2-%23). Archived from the original (http://www.cs.rpi.edu/~musser/gp/introsort.ps) on 7 March 2023.

- Niklaus Wirth. *Algorithms and Data Structures*. Prentice-Hall, Inc., 1985. ISBN 0-13-022005-1.
